SIMATS ENGINEERING

ASSESSMENT TOOL 1: Database Design and Connectivity

COURSE NAME: Query Processing for Data Science **COURSE CODE:** DSA0501

COURSE OUTCOME COVERED

CO2: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete.

Assessment Tool Weightage: 40%

Total Marks: 100

Student Name: Answar Vijes D **Register No: 192324028**

Code of Conduct

I Answar Vijes D / 192324028 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: _

Database Design and Connectivity Rubric

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Total Marks Awarded:

Answer Sheet - Database Design and Connectivity

Course: Query Processing for Data Science (DSA0501) | Student: Answar vijes D | Reg No: 192324028

Notes: All programs are written in Python 3. The question paper does not supply input data, so each answer uses small sample data created inside the program. The outputs shown were produced by actually running the code.

1. Student Database Design and Connectivity (10 Marks)

Task: Design an SQLite Student table with a primary key. Connect with Python, create the table if it does not exist, insert at least five records and display all records.

Approach: Requirement: store student records uniquely. RollNo is the primary key; constraints (NOT NULL, UNIQUE email) keep the data valid. The table is in 3NF because every non-key column depends only on RollNo.

Database design (ER diagram and table structure):

```
ER Diagram:       [ STUDENT ]  (entity)
                     |-- RollNo (PK)
                     |-- Name, Department, Year, Email, CGPA

Table  Student
 RollNo     INTEGER  PRIMARY KEY
 Name       TEXT     NOT  NULL
 Department TEXT     NOT NULL
 Year       INTEGER  CHECK (Year BETWEEN 1 AND 4)
 Email      TEXT     UNIQUE
 CGPA       REAL
```

Program:

```
import sqlite3, os
if os.path.exists("college.db"): os.remove("college.db") # start fresh each run
con = sqlite3.connect("college.db")                  # connect to SQLite databas
ecur = con.cursor()

cur.execute("""CREATE TABLE IF NOT EXISTS Student(
    RollNo INTEGER PRIMARY KEY,
    Name TEXT NOT NULL,
    Department TEXT NOT NULL,
    Year INTEGER CHECK (Year BETWEEN 1 AND 4),
    Email TEXT UNIQUE,
    CGPA REAL)""")

students = [
    (101, "Aarthi",  "CSE",  2, "aarthi@college.edu", 8.9),
    (102, "Bharath", "ECE",  3, "bharath@college.edu", 7.8),
    (103, "Chandru", "CSE",  1, "chandru@college.edu", 9.1),
    (104, "Deepa",   "MECH", 4, "deepa@college.edu",  8.2),
    (105, "Elango",  "IT",   2, "elango@college.edu", 7.5),
]
cur.executemany("INSERT INTO Student VALUES (?,?,?,?,?,?)", students)
con.commit()

cur.execute("SELECT * FROM Student")
print("RollNo | Name | Department | Year | Email | CGPA")
for row in cur.fetchall():
    print(row)
con.close()
```

Output:

```
RollNo | Name | Department | Year | Email | CGPA
(101, 'Aarthi', 'CSE', 2, 'aarthi@college.edu', 8.9)
(102, 'Bharath', 'ECE', 3, 'bharath@college.edu', 7.8)
(103, 'Chandru', 'CSE', 1, 'chandru@college.edu', 9.1)
(104, 'Deepa', 'MECH', 4, 'deepa@college.edu', 8.2)
(105, 'Elango', 'IT', 2, 'elango@college.edu', 7.5)
```

2. Library Management Database (10 Marks)

Task: Design a Book table (Book ID, Title, Author, Publisher, Price). Create the table, insert sample records, update the price of a selected book and retrieve the updated information.
Approach: BookID is the primary key; Price has a CHECK constraint so it cannot be negative. An UPDATE with a WHERE clause changes only the selected book.

Database design (ER diagram and table structure):

```
ER Diagram:      [ BOOK ]
                    |-- BookID (PK)
                    |-- Title, Author, Publisher, Pric

Table Boo
 kBookID  INTEGER  PRIMARY KEY
 Title    TEXT     NOT NULL
 Author   TEXT     NOT NULL
 Publisher TEXT
 Price    REAL     CHECK (Price >= 0)
```

Program:

```
import sqlite3, os
if os.path.exists("library.db"): os.remove("library.db")  # start fresh each run
con = sqlite3.connect("library.db")                  # connect to SQLite databas
ecur = con.cursor()

cur.execute("""CREATE TABLE Book(
    BookID INTEGER PRIMARY KEY, Title TEXT NOT NULL, Author TEXT NOT NULL,
    Publisher TEXT, Price REAL CHECK (Price >= 0))""")

books = [
    (1, "Python Crash Course", "Eric Matthes",   "No Starch Press", 599.0),
    (2, "Database Systems",    "Elmasri",        "Pearson",         725.0),
    (3, "Data Wrangling",      "Jacqueline Kazil","O'Reilly",    650.0),
    (4, "Clean Code",          "Robert Martin", "Prentice Hall",   540.0),
]
cur.executemany("INSERT INTO Book VALUES (?,?,?,?,?)", books)

print("Before update:", cur.execute("SELECT BookID, Title, Price FROM Book WHERE BookID=3").fetchone
()) cur.execute("UPDATE Book SET Price = ? WHERE BookID = ?", (700.0, 3))
con.commit()

print("After update :", cur.execute("SELECT BookID, Title, Price FROM Book WHERE BookID=3").fetcho
ne()) print("\nAll books:")
for r in cur.execute("SELECT * FROM Book")
    :print(r)
con.close()
```

Output:

```
Before update: (3, 'Data Wrangling', 650.0
)After update : (3, 'Data Wrangling', 700.
0)

All books:
(1, 'Python Crash Course', 'Eric Matthes', 'No Starch Press', 599.
0)(2, 'Database Systems', 'Elmasri', 'Pearson', 725.0)
(3, 'Data Wrangling', 'Jacqueline Kazil', "O'Reilly", 700.0
```

3. Employee and Department Database Design (10 Marks)

Task: Design Department and Employee tables with primary and foreign keys. Create both, insert sample data and display employee name with department name using an SQL JOIN.
Approach: One department has many employees (1:N). DeptID is the PK of Department and a FK in Employee. This removes repetition of department data (2NF/3NF).

Database design (ER diagram and table structure):

```
ER Diagram:   [ DEPARTMENT ] 1 ----< works in >---- N [ EMPLOYEE ]

Table Department              Table Employee
 DeptID   INTEGER  PK         EmpID   INTEGER PK
 DeptName  TEXT NOT NULL     EmpName TEXT NOT NULL
 Location  TEXT              Salary  REAL
                             DeptID  INTEGER FK -> Department(DeptID)
```

Program:

```
import sqlite3, os
if os.path.exists("company.db"): os.remove("company.db")# start fresh each run
con = sqlite3.connect("company.db")                # connect to SQLite databas
ecur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON")
cur.execute("CREATE TABLE Department(DeptID INTEGER PRIMARY KEY, DeptName TEXT NOT NULL, Location TEXT)")
cur.execute("""CREATE TABLE Employee(
    EmpID INTEGER PRIMARY KEY, EmpName TEXT NOT NULL, Salary REAL,
    DeptID INTEGER, FOREIGN KEY (DeptID) REFERENCES Department(DeptID))""")

cur.executemany("INSERT INTO Department VALUES (?,?,?)",
    [(10, "IT", "Chennai"), (20, "HR", "Bengaluru"), (30, "Finance", "Mumbai")])
cur.executemany("INSERT INTO Employee VALUES (?,?,?,?)",
    [(1, "Arun", 65000, 10), (2, "Bhavana", 48000, 20), (3, "Charan", 52000, 30),
     (4, "Divya", 72000, 10), (5, "Elango", 45000, 20)])
con.commit()

query = """SELECT e.EmpName, d.DeptName
           FROM Employee e JOIN Department d ON e.DeptID = d.DeptID
           ORDER BY e.EmpID"""
print("Employee Name| Department")
for name, dept in cur.execute(query):
    print(f"{name:<14} | {dept}")
con.close()
```

Output:

```
Employee Name| Department
Arun          | IT
Bhavana       | HR
Charan        | Finance
Divya         | IT
Elango        | HR
```

4. Hospital Patient Database (10 Marks)

Task: Design a Patient table with constraints. Insert patients, update the contact number of a specified patient, delete a discharged patient's record and display the remaining records.

Approach: PatientID is the PK, Age has a CHECK constraint, Contact is UNIQUE. UPDATE and DELETE use WHERE so that only the intended patient is affected.

Database design (ER diagram and table structure):

```
ER Diagram:      [ PATIENT ]
                    |-- PatientID (PK)
                    |-- Name, Age, Gender, Disease, Contact, Status

Table Patient
 PatientID INTEGER PRIMARY KEY
 Name      TEXT    NOT NULL
 Age       INTEGER CHECK (Age BETWEEN 0 AND 120)
 Gender    TEXT    CHECK (Gender IN ('M','F'))
 Disease   TEXT
 Contact   TEXT    UNIQUE
 Status    TEXT    DEFAULT 'Admitted'
```

Program:

```
import sqlite3, os
if os.path.exists("hospital.db"): os.remove("hospital.db") # start fresh each run
con = sqlite3.connect("hospital.db")                  # connect to SQLite databas
ecur = con.cursor()

cur.execute("""CREATE TABLE Patient(
    PatientID INTEGER PRIMARY KEY, Name TEXT NOT NULL,
    Age INTEGER CHECK (Age BETWEEN 0 AND 120),
    Gender TEXT CHECK (Gender IN ('M','F')), Disease TEXT,
    Contact TEXT UNIQUE, Status TEXT DEFAULT 'Admitted')"""
    )

    (1, "Kamala",  67, "F", "Diabetes",   "9876500001", "Admitted"),
    (2, "Rajesh",  45, "M", "Fracture",   "9876500002", "Discharged")
    (3, "Sundar",  72, "M", "Hypertension","9876500003", "Admitted"),
    (4, "Priya",   34, "F", "Fever",      "9876500004", "Admitted"),
])

cur.execute("UPDATE Patient SET Contact = ? WHERE PatientID = ?", ("9999988888", 3
))print("Contact updated for PatientID 3, rows affected:", cur.rowcount)

cur.execute("DELETE FROM Patient WHERE Status = 'Discharged'")
print("Discharged patients deleted, rows affected:", cur.rowcount)
con.commit()

print("\nRemaining records:")
for r in cur.execute("SELECT * FROM Patient"
    ):print(r)
con.close()
```

Output:

```
Contact updated for PatientID 3, rows affected: 1
Discharged patients deleted, rows affected: 1

Remaining records:
(1, 'Kamala', 67, 'F', 'Diabetes', '9876500001', 'Admitted')
(3, 'Sundar', 72, 'M', 'Hypertension', '9999988888', 'Admitted')
(4, 'Priya', 34, 'F', 'Fever', '9876500004', 'Admitted')
```

5. Online Course Registration System (10 Marks)

Task: Design Student and Course_Registration tables with primary and foreign keys. Insert sample records and retrieve student names with the courses they registered for.

Approach: A student can register for many courses (1:N). Course_Registration holds RegID (PK) and StudentID (FK).

Database design (ER diagram and table structure):

```
ER Diagram:  [ STUDENT ] 1 ----< registers >---- N [ COURSE_REGISTRATION ]

Table  Student            Table Course_Registration
 StudentID INTEGER PK      RegID       INTEGER PK
 Name      TEXT NOT NULL   StudentID   INTEGER FK -> Student(StudentID)
 Email     TEXT UNIQUE     CourseName  TEXT NOT NULL
                           RegDate     TEXT
```

Program:

```
import sqlite3, os
if os.path.exists("elearning.db"): os.remove("elearning.db       # start fresh each run
con = sqlite3.connect("elearning.db")                    # connect to SQLite databas
ecur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON")
cur.execute("CREATE TABLE Student(StudentID INTEGER PRIMARY KEY, Name TEXT NOT NULL, Email TEXT UNIQUE)")
cur.execute("""CREATE TABLE Course_Registration(
    RegID INTEGER PRIMARY KEY, StudentID INTEGER NOT NULL,
    CourseName TEXT NOT NULL, RegDate TEXT,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID))""")

cur.executemany("INSERT INTO Student VALUES (?,?,?)", [
    (1, "Aarthi", "aarthi@mail.com"), (2, "Bharath", "bharath@mail.com"), (3, "Chandru", "chandru@mail.com")])
cur.executemany("INSERT INTO Course_Registration VALUES (?,?,?,?)", [
    (1, 1, "Python Programming", "2025-06-01"), (2, 1, "Data Science", "2025-06-03"),
    (3, 2, "Machine Learning",   "2025-06-05"), (4, 3, "Python Programming", "2025-06-07"),
    (5, 3, "SQL Fundamentals",   "2025-06-08")])
con.commit()

q = """SELECT s.Name, c.CourseName FROM Student s
       JOIN Course_Registration c ON s.StudentID = c.StudentID
       ORDER BY s.Name"""
print("Student   | Course")
for name, course in cur.execute(q):
    print(f"{name:<9} | {course}")
con.close()
```

Output:

```
Student   | Course
Aarthi    | Python Programming
Aarthi    |  Data  Science
Bharath   | Machine Learning
Chandru   | Python Programming
Chandru   | SQL Fundamentals
```

6. Banking Account Management System (10 Marks)

Task: Design an Account table with constraints. Insert customer account details, update the balance of a specified customer and display all customer records.

Approach: AccountNo is the PK; Balance cannot be negative (CHECK) and AccountType is restricted to valid values.

Database design (ER diagram and table structure):

```
ER Diagram:       [ ACCOUNT ]
                     |-- AccountNo (PK)
                     |-- CustomerName, AccountType, Balance, Branch

Table Account
 AccountNo  INTEGER PRIMARY KEY
 CustomerNam   TEXT    NOT NULL
 eAccountTyp   TEXT    CHECK (AccountType IN ('Savings','Current')
 eBalance      REAL    )CHECK (Balance >= 0)
 Branch        TEXT
```

Program:

```
import sqlite3, os
if os.path.exists("bank.db"): os.remove("bank.db") #  start  fresh  each  run
con = sqlite3.connect("bank.db")                  # connect to SQLite databas
ecur = con.cursor()

cur.execute("""CREATE TABLE Account(
    AccountNo INTEGER PRIMARY KEY, CustomerName TEXT NOT NULL,
    AccountType TEXT CHECK (AccountType IN ('Savings','Current')),
    Balance REAL CHECK (Balance >= 0), Branch TEXT)""")

cur.executemany("INSERT INTO Account VALUES (?,?,?,?,?)", [
    (1001, "Arun",    "Savings", 25000.00, "Chennai"),
    (1002, "Bhavana", "Current", 120000.00, "Madurai"),
    (1003, "Charan",  "Savings", 8500.50,  "Salem"),
    (1004, "Divya",   "Savings", 43000.00, "Trichy"),
])

cur.execute("UPDATE Account SET Balance = Balance + ? WHERE CustomerName = ?", (5000, "Charan")
)con.commit()
print("Deposited 5000 into Charan's account\n")

print("AccountNo | Customer | Type | Balance | Branch"
)for r in cur.execute("SELECT * FROM Account"):
    print(r)

try:                                                  # constraint demo
    cur.execute("UPDATE Account SET Balance = -100 WHERE AccountNo = 100
1")except sqlite3.IntegrityError as e:
    print("\nRejected by CHECK constraint:", e)
con.close()
```

Output:

```
Deposited 5000 into Charan's account

AccountNo | Customer | Type | Balance | Branc
h(1001, 'Arun', 'Savings', 25000.0, 'Chennai'
)
(1002, 'Bhavana', 'Current', 120000.0, 'Madurai')
(1003, 'Charan', 'Savings', 13500.5, 'Salem')
(1004, 'Divya', 'Savings', 43000.0, 'Trichy')

Rejected by CHECK constraint: CHECK constraint failed: Balance >= 0
```

7. Inventory Management System (10 Marks)

Task: Design a Product table (Product ID, Name, Category, Quantity, Unit Price). Insert products, identify products with quantity less than 10, update their stock and display the updated inventory.
Approach: A WHERE Quantity < 10 query finds low-stock items; one UPDATE restocks all of them and the final SELECT shows the new stock.

Database design (ER diagram and table structure):

```
ER Diagram:       [ PRODUCT ]
                     |-- ProductID (PK)
                     |-- ProductName, Category, Quantity, UnitPrice

Table Product
 ProductID INTEGER PRIMARY KEY
 ProductName TEXT    NOT NULL
 Category  TEXT
 Quantity  INTEGER CHECK (Quantity >= 0)
 UnitPrice   REAL    CHECK (UnitPrice > 0)
```

Program:

```
import sqlite3, os
if os.path.exists("inventory.db"): os.remove("inventory.db"      # start fresh each run
con = sqlite3.connect("inventory.db")                  # connect to SQLite databas
ecur = con.cursor()

cur.execute("""CREATE TABLE Product(
    ProductID INTEGER PRIMARY KEY, ProductName TEXT NOT NULL, Category TEXT,
    Quantity INTEGER CHECK (Quantity >= 0), UnitPrice REAL CHECK (UnitPrice > 0))""")

cur.executemany("INSERT INTO Product VALUES (?,?,?,?,?)", [
    (1, "Notebook",  "Stationery",  120,  45.0),
    (2, "Pen Pack",  "Stationery",    6,  60.0),
    (3, "Mouse",     "Electronics"    4, 450.0),
    (4, "Keyboard",  ,"Electronics   15, 900.0),
                     ",               9, 150.0),
])

print("Products with quantity < 10:")
for r in cur.execute("SELECT ProductID, ProductName, Quantity FROM Product WHERE Quantity < 10"):
    print(r)

cur.execute("UPDATE Product SET Quantity = Quantity + 50 WHERE Quantity < 10")   # restock
con.commit()
print("\nRestocked rows:", cur.rowcount)

print("\nUpdated inventory:")
for r in cur.execute("SELECT * FROM Product"
    ):print(r)
con.close()
```

Output:

```
Products with quantity < 10:
(2, 'Pen Pack', 6)
(3, 'Mouse', 4)
(5, 'USB Cable', 9)

Restocked rows: 3

Updated inventory:
(1, 'Notebook', 'Stationery', 120, 45.0)
(2, 'Pen Pack', 'Stationery', 56, 60.0)
(3, 'Mouse', 'Electronics', 54, 450.0)
(4, 'Keyboard', 'Electronics', 15, 900.0)
(5, 'USB Cable', 'Electronics', 59, 150.0)
```

8. Course and Faculty Management System (10 Marks)

Task: Design Faculty and Course tables with primary and foreign keys. Insert sample records and display each faculty member with the courses assigned using an SQL JOIN.

Approach: One faculty member can teach many courses (1:N); FacultyID is the FK in Course.

Database design (ER diagram and table structure):

```
ER Diagram:  [ FACULTY ] 1 ----< teaches >---- N [ COURSE ]

Table   Faculty           Table Course
 FacultyID INTEGER PK      CourseID     INTEGER PK
 FacultyName TEXT NOT NULL CourseName TEXT NOT NULL
 Department TEXT           Credits      INTEGER
                           FacultyID    INTEGER FK -> Faculty(FacultyID)
```

Program:

```
import sqlite3, os
if os.path.exists("university.db"): os.remove("university.d      # start fresh each run
con = sqlite3.connect("university.db")                    # connect to SQLite databas
ecur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON")
cur.execute("CREATE TABLE Faculty(FacultyID INTEGER PRIMARY KEY, FacultyName TEXT NOT NULL, Department TEXT)")
cur.execute("""CREATE TABLE Course(
    CourseID INTEGER PRIMARY KEY, CourseName TEXT NOT NULL, Credits INTEGER,
    FacultyID INTEGER, FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID))""")

cur.executemany("INSERT INTO Faculty VALUES (?,?,?)", [
    (1, "Dr. Ramesh", "CSE"), (2, "Dr. Sulochana", "ECE"), (3, "Dr. Karthik", "IT")])
cur.executemany("INSERT INTO Course VALUES (?,?,?,?)", [
    (501, "Data Structures", 4, 1), (502, "Database Systems", 3, 1),
    (503, "Digital Electronics", 4, 2), (504, "Python Programming", 3, 3),
    (505, "Machine Learning", 4, 3)])
con.commit()

q = """SELECT f.FacultyName, c.CourseName FROM Faculty f
       JOIN Course c ON f.FacultyID = c.FacultyID ORDER BY f.FacultyName""
"print("Faculty      | Course")
for fac, course in cur.execute(q)
    :print(f"{fac:<14} | {course}"
    )
```

Output:

```
Faculty        | Course
Dr. Karthik    | Python Programming
Dr. Karthik    | Machine Learning
Dr. Ramesh     | Data Structures
Dr. Ramesh     | Database Systems
Dr. Sulochana | Digital Electronics
```

9. Vehicle Registration Database (10 Marks)

Task: Design a Vehicle table with constraints. Insert vehicle records, search for a vehicle using its registration number and display the retrieved information.

Approach: RegNo is the PK so every vehicle is unique and the search by registration number is fast. A parameterised query prevents SQL injection.

Database design (ER diagram and table structure):

```
ER Diagram:      [ VEHICLE ]
                    |-- RegNo (PK)
                    |-- OwnerName, VehicleType, Model, RegDate

Table Vehicle
 RegNo        TEXT PRIMARY KEY
 OwnerName    TEXT NOT NULL
 VehicleType TEXT CHECK (VehicleType IN ('Car','Bike','Truck','Bus')
 )Model       TEXT
 RegDate      TEXT
```

Program:

```
import sqlite3, os
if os.path.exists("transport.db"): os.remove("transport.d      # start fresh each run
con = sqlite3.connect("transport.db")                  # connect to SQLite databas
ecur = con.cursor()

cur.execute("""CREATE TABLE Vehicle(
    RegNo TEXT PRIMARY KEY, OwnerName TEXT NOT NULL,
    VehicleType TEXT CHECK (VehicleType IN ('Car','Bike','Truck','Bus')),
    Model TEXT, RegDate TEXT)""")

cur.executemany("INSERT INTO Vehicle VALUES (?,?,?,?,?)", [
    ("TN01AB1234", "Arun","Car",  "Hyundai i20",    "2022-03-15"),
    ("TN09CD5678", "Bhavana", "Bike","Honda Activa",    "2023-07-21"),
    ("TN22EF9012", "Charan","Truck", "Tata 407",        "2021-11-02"),
    ("TN33GH3456", "Divya",  "Car",   "Maruti Swift",    "2024-01-30"),
])
con.commit()

reg = "TN09CD5678"
row = cur.execute("SELECT * FROM Vehicle WHERE RegNo = ?", (reg,)).fetchone
()print("Search for:", reg)
if row:
    for label, val in zip(["RegNo", "Owner", "Type", "Model", "Reg Date"], ro
        w):print(f"    {label:<9}: {val}")
else:
    print("  Vehicle not found")

print("\nSearch for TN00XX0000:",
      cur.execute("SELECT * FROM Vehicle WHERE RegNo = ?", ("TN00XX0000",)).fetchone())
con.close()
```

Output:

```
Search for: TN09CD5678
  RegNo    : TN09CD5678
  Owner    : Bhavana
  Type     : Bike
  Model    : Honda Activa
  Reg Date : 2023-07-21

Search for TN00XX0000: None
```

10. Hospital Appointment Management System (10 Marks)

Task: Design Doctors and Appointments tables with primary and foreign keys. Insert sample records and display the doctor name with the appointment details using a JOIN.
Approach: One doctor has many appointments (1:N); DoctorID is the FK in Appointments.

Database design (ER diagram and table structure):

```
ER Diagram:  [ DOCTORS ] 1 ----< attends >---- N [ APPOINTMENTS ]

Table Doctors                  Table  Appointments
 DoctorID  INTEGER PK           AppointmentID INTEGER PK
 DoctorName TEXT NOT NULL       PatientName  TEXT NOT NULL
 Speciality TEXT                AppDate      TEXT
                                AppTime      TEXT
                                DoctorID     INTEGER FK -> Doctors(DoctorID)
```

Program:

```
import sqlite3, os
if os.path.exists("appointments.db"): os.remove("appointments.d       # start fresh each run
con = sqlite3.connect("appointments.db")                    # connect to SQLite database
cur = con.cursor()

cur.execute("PRAGMA foreign_keys = ON")
cur.execute("CREATE TABLE Doctors(DoctorID INTEGER PRIMARY KEY, DoctorName TEXT NOT NULL, Speciality TEXT)")
cur.execute("""CREATE TABLE Appointments(
    AppointmentID INTEGER PRIMARY KEY, PatientName TEXT NOT NULL,
    AppDate TEXT, AppTime TEXT, DoctorID INTEGER,
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID))""")

cur.executemany("INSERT INTO Doctors VALUES (?,?,?)", [
    (1, "Dr. Anand", "Cardiology"), (2, "Dr. Meena", "Pediatrics"), (3, "Dr. Rao", "Orthopedics")])
cur.executemany("INSERT INTO Appointments VALUES (?,?,?,?,?)", [
    (901, "Kamala",  "2025-07-01", "10:00", 1), (902, "Rajesh", "2025-07-01", "11:30", 3),
    (903, "Baby Ria","2025-07-02", "09:30", 2), (904, "Sundar", "2025-07-03", "14:00", 1)])
con.commit()

q = """SELECT d.DoctorName, d.Speciality, a.PatientName, a.AppDate, a.AppTime
       FROM Doctors d JOIN Appointments a ON d.DoctorID = a.DoctorID
       ORDER BY a.AppDate, a.AppTime"""
print(f"{'Doctor':<11}{'Speciality':<13}{'Patient':<10}{'Date':<12}Tim
e") for r in cur.execute(q):
    print(f"{r[0]:<11}{r[1]:<13}{r[2]:<10}{r[3]:<12}{r[4]}")
con.close()
```

Output:

```
Doctor     Speciality   Patient   Date        Time
 Dr. Anand Cardiology   Kamala    2025-07-01  10:00
Dr. Rao    Orthopedics  Rajesh    2025-07-01  11:30
 Dr. Meena Pediatrics   Baby Ria  2025-07-02  09:30
 Dr. Anand Cardiology   Sundar    2025-07-03  14:00
```